

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 1

The OS as a Resource Manager

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. Files and directories **2**

1.1 Creating, copying and removing files 2

1.2 Ownership and permissions 2

1.3 The root of the tree 3

2 Part 2. Processes **3**

2.1 The processes the OS is juggling 3

2.2 Starting and stopping a background process 3

2.3 A process seen through /proc 4

3 Part 3. Memory **4**

4 Part 4. Devices and storage **5**

Conclusions **6**

Purpose of the work

The purpose of this lab is to watch the operating system do its four core jobs (managing files, processes, memory and devices) directly from a Linux terminal, and to describe what each command reveals about the OS rather than only which command was typed.

The work was done on my own laptop running Arch Linux instead of the course VM, so some numbers below (RAM, number of processes, disks) are larger than they would be on the VM.

```
$ whoami
Nico
$ uname -a
Linux Nico 7.0.12-arch1-1 #1 SMP PREEMPT_DYNAMIC Wed, 10 Jun 2026 08:58:02 +0000 x86_64 GNU/Linux
$ uptime
 15:42:21 up  1:15,  1 user,  load average: 3.11, 2.50, 2.34
```

The kernel managing this machine is Linux 7.0.12 for x86_64, and it had been running for 1 hour and 15 minutes.

1 Part 1. Files and directories

1.1 Creating, copying and removing files

```
$ cd ~/os-lab1 && mkdir demo && cd demo
$ echo "hello operating systems" > note.txt
$ cp note.txt copy.txt
$ mv copy.txt renamed.txt
$ ls -l
total 8
-rw-r--r-- 1 Nico Nico 24 Oct  5 15:42 note.txt
-rw-r--r-- 1 Nico Nico 24 Oct  5 15:42 renamed.txt
$ rm renamed.txt
```

1.2 Ownership and permissions

```
$ ls -l note.txt
-rw-r--r-- 1 Nico Nico 24 Oct  5 15:42 note.txt
$ chmod 600 note.txt
$ ls -l note.txt
-rw------- 1 Nico Nico 24 Oct  5 15:42 note.txt
$ chmod 644 note.txt
$ id
uid=1000(Nico) gid=1000(Nico) groups=1000(Nico),958(ollama),961(docker),983(video),...
```

1.3 The root of the tree

```
$ ls -la /
lrwxrwxrwx   1 root root     7 Oct 12  2025 bin -> usr/bin
drwxr-xr-x   4 root root  4096 Sep 21 21:17 boot
drwxr-xr-x  22 root root  4540 Oct  5 15:12 dev
drwxr-xr-x  92 root root  4096 Oct  5 15:21 etc
drwxr-xr-x   5 root root  4096 Apr  3  2026 home
lrwxrwxrwx   1 root root     7 Oct 12  2025 lib -> usr/lib
dr-xr-xr-x 560 root root     0 Oct  5  2026 proc
lrwxrwxrwx   1 root root     7 Oct 12  2025 sbin -> usr/bin
drwxrwxrwt  49 root root  1300 Oct  5 15:42 tmp
drwxr-xr-x  10 root root  4096 Sep 30 14:02 usr
```

Observations:

1. **Owner and group.** The files belong to my user `Nico` and to the group `Nico`. Arch creates a private group with the same name for every user (`gid=1000(Nico)`), and new files get the user's primary group.
2. **The ten characters after `chmod 600`.** `-rw-------`: the first `-` means a regular file (`d` would be a directory, `l` a symbolic link, `c`/`b` a device); `rw-` means the owner can read and write but not execute; the two `---` groups mean the group and everyone else have no rights at all. The value 600 is octal: 6 = 4 (read) + 2 (write) for the owner, 0 for the group, 0 for others. After `chmod 644` the file is `-rw-r--r--` again.
3. **Two directories under `/`.** `/etc` holds the system-wide configuration files (`passwd`, `fstab`, `hostname`, `pacman.conf`). `/proc` is not stored on disk at all: the kernel generates it on the fly, with one folder per running process, which is why its size is 0.

Interesting detail: on Arch, `bin`, `sbin` and `lib` are not real directories but symbolic links into `/usr`.

2 Part 2. Processes

2.1 The processes the OS is juggling

```
$ ps aux | head -4
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  26424 14864 ?        Ss   14:27   0:01 /usr/lib/systemd/systemd --
    ↪ switched-root --system --deserialize=55 splash
root           2  0.0  0.0      0     0 ?        S    14:27   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    14:27   0:00 [pool_workqueue_release]
$ ps aux | wc -l
501
$ top -b -n 1 | head -3
top - 15:42:29 up  1:15,  1 user,  load average: 3.03, 2.50, 2.35
Tasks: 500 total, 1 running, 498 sleep, 1 d-sleep, 0 stopped, 0 zombie
```

2.2 Starting and stopping a background process

```
$ sleep 300 &
[1] 239197
```

```
$ jobs
[1]+  Running                 sleep 300 &
$ ps -ef | grep sleep
Nico      239197  239194  0 15:42 ?        00:00:00 sleep 300
$ kill 239197
$ jobs
$
```

2.3 A process seen through /proc

```
$ sleep 300 & PID=$!
$ cat /proc/$PID/status | head -8
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   239201
Ngid:   0
Pid:    239201
PPid:   239194
TracerPid:      0
$ kill $PID
```

Observations:

1. **PID 1.** It is `systemd` (`/usr/lib/systemd/systemd`), the init system. It is the first user-space process the kernel starts at boot. It starts and supervises all services, it is the ancestor of every other user process, and it adopts orphaned processes whose parent has died. PID 2, `kthreadd`, is the parent of the kernel's own threads.
2. **How many processes.** `ps aux | wc -l` printed 501; one line is the header, so about 500 processes existed, which `top` confirms (`Tasks: 500 total`). About 300 of them are kernel threads (the ones in `[brackets]`, with VSZ 0); the rest are my desktop session and system services. Almost all of them were sleeping.
3. **The `State:` line.** `S (sleeping)`: the process is in an interruptible sleep, waiting for its timer to expire, and receives no CPU time until then. Its `PPid` is the shell that started it, so the shell is its parent.

3 Part 3. Memory

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:            30Gi         9Gi        14Gi       479Mi       6.5Gi        20Gi
Swap:             0B          0B          0B
$ cat /proc/meminfo | head -6
MemTotal:       32124248 kB
MemFree:        15344088 kB
MemAvailable:   21642580 kB
Buffers:          867072 kB
Cached:          5035328 kB
SwapCached:            0 kB
$ sleep 300 & PID=$!
$ grep -E 'VmSize|VmRSS|RssAnon|RssFile' /proc/$PID/status
```

```
VmSize:     3216 kB
VmRSS:      2036 kB
RssAnon:     112 kB
RssFile:    1924 kB
$ kill $PID
```

Observations:

1. **Total and free RAM.** The machine has 30 GiB of RAM (`MemTotal` 32 124 248 kB). 14 GiB is completely free, but 20 GiB is *available*: the kernel uses otherwise idle RAM as a disk cache (`buff/cache` 6.5 GiB) and drops that cache as soon as a program needs the memory.
2. **Swap.** Swap is a partition or file on disk where the kernel moves memory pages that have not been used for a while when RAM fills up. This frees RAM for other processes, but those pages become slow to access again. On this laptop no swap is configured (`Swap: 0B`), so when memory runs out the kernel's OOM killer has to end a process instead.
3. **VmRSS of `sleep`.** A bare `sleep` keeps 2036 kB (≈2 MB) resident in RAM, out of 3216 kB of virtual memory. This surprised me at first, but the breakdown explains it: only `RssAnon` = 112 kB is the process's own private memory, while `RssFile` = 1924 kB is the mapped code of the `sleep` binary and of the C library. Those pages are shared with every process that uses libc, so they exist in physical RAM only once. VmSize is larger than VmRSS because the OS gives a page real RAM only when it is first touched (demand paging).

4 Part 4. Devices and storage

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p5  192G  152G   31G  84% /
devtmpfs         16G     0   16G   0% /dev
tmpfs            16G   11M   16G   1% /dev/shm
tmpfs            16G  131M   16G   1% /tmp
/dev/nvme0n1p1   96M   86M   11M  90% /boot/efi
$ lsblk -i
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 953.9G  0 disk
|-nvme0n1p1 259:1    0   100M  0 part /boot/efi
|-nvme0n1p2 259:2    0    16M  0 part
|-nvme0n1p3 259:3    0 247.6G  0 part
|-nvme0n1p4 259:4    0 510.8G  0 part
`-nvme0n1p5 259:5    0 195.3G  0 part /
$ du -sh ~/os-lab1
12K     /home/Nico/os-lab1
$ ls -l /dev/null /dev/nvme0n1 /dev/tty /dev/zero /dev/random
crw-rw-rw- 1 root root   1, 3 Oct  5  2026 /dev/null
brw-rw---- 1 root disk 259, 0 Oct  5  2026 /dev/nvme0n1
crw-rw-rw- 1 root root   1, 8 Oct  5  2026 /dev/random
crw-rw-rw- 1 root tty    5, 0 Oct  5 15:38 /dev/tty
crw-rw-rw- 1 root root   1, 5 Oct  5  2026 /dev/zero
$ mount | head -3
/dev/nvme0n1p5 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=15872424k,nr_inodes=3968106,mode=755,inode64,huge=
    ↪ within_size)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,huge=within_size,usrquota)
```

Observations:

1. **The root filesystem.** `/` is mounted from `/dev/nvme0n1p5`, partition 5 of the NVMe SSD `nvme0n1` (953.9 GB). It is an ext4 filesystem of 192 GB, 84% used.
2. **One entry from `/dev`.** `/dev/nvme0n1` is the whole SSD. The leading `b` marks a *block device*, read and written in fixed-size blocks, and instead of a size `ls` shows the numbers `259, 0`: the major number selects the driver and the minor number selects the device. `/dev/null` is a *character device* (`c`, numbers `1, 3`) that is not hardware at all: it discards everything written to it.
3. **"Everything is a file".** The kernel presents disks, terminals, the random number generator and even processes (`/proc`) as entries in the same file tree, so the same tools (`ls`, `cat`) and the same system calls (`open`, `read`, `write`) work on all of them.

Interesting detail: `mount` shows that `/dev`, `/dev/shm` and `/tmp` are `devtmpfs`/`tmpfs` filesystems, which live only in RAM and disappear at reboot.

Conclusions

The operating system manages four resources: files, processes, memory and devices. I observed files with `ls -l` and `chmod`, which showed that every file has an owner, a group and permission bits the kernel enforces. I observed processes with `ps aux` and `kill`: about 500 of them, almost all sleeping, all descending from `systemd` as PID 1. Memory, seen with `free -h` and `/proc`, showed 30 GiB of RAM shared between processes, no swap, and code pages that are shared rather than copied. Devices, seen with `lsblk` and `df -h`, showed the SSD and its partitions presented as files under `/dev`, with `/` mounted from `nvme0n1p5`. Each command is a different view of the same kernel, which shares these resources between programs and protects them from each other.